Python Engineering

Ejercicio 05 - Bucles: Suma de cuadrados

Teoría + modelo mental + trazado paso a paso + tests + seguimiento del aprendizaje

Objetivo del ejercicio: comprender la iteración mediante cuatro ideas fundamentales: contador, acumulador, estado y bucle, y conectarlas con la futura integración numérica de problemas dinámicos de ingeniería.

1. El problema matemático

Queremos calcular la suma de los cuadrados de todos los enteros desde 1 hasta N:

$$S = \sum_{i=1}^{N} i^2$$

Para N = 4, el cálculo es:

$$S = 1^2 + 2^2 + 3^2 + 4^2 = 30$$

En este ejercicio deliberadamente no utilizamos la expresión cerrada. El objetivo es hacer que el ordenador ejecute explícitamente la operación repetida.

> **Idea clave: cuando la misma operación debe aplicarse a una secuencia de valores, aparece de forma natural una iteración.**

2. ¿Qué cambia respecto a los ejercicios anteriores?

Hasta ahora, la mayoría de programas seguían un camino corto: entrada → cálculo → salida. Aquí una operación se repite para 1, 2, 3, ..., N.

```
1^2
2^2
3^2
4^2
...
N^2
```

Un bucle necesita una forma de saber qué elemento se está procesando. En este ejercicio ese papel lo desempeña i.

El contador

La variable i cambia en cada pasada del bucle: 1, 2, 3, ..., N. Es el contador o índice de la iteración actual.

3. Acumulador y estado del programa

El programa debe recordar la suma parcial de las iteraciones anteriores. Por ello inicializamos una variable, por ejemplo total = 0, y la actualizamos en cada pasada.

$$total_{new} = total_{old} + i^2$$

Esta variable se denomina acumulador. Su valor contiene información producida por las iteraciones anteriores.

#	i	total
inicial	—	0
1	1	1
2	2	5
3	3	14
4	4	30

El estado es la información actual almacenada por el algoritmo. Más adelante la misma idea aparecerá como x(k+1) = f(x(k)) en simulación numérica.

Conexión con dinámica de ingeniería

El ejercicio de suma de cuadrados parece sencillo, pero su patrón de actualización es estructuralmente similar al de una simulación temporal:

$$v_{k+1} = v_k + a_k \, \Delta t$$

$$h_{k+1} = h_k + v_k \, \Delta t$$

En ambos casos el siguiente estado se construye a partir del estado anterior.

4. Elegir un bucle: for vs while

Aquí sabemos de antemano cuántos valores deben procesarse: desde 1 hasta N. Por tanto, un bucle for es la elección natural.

```
Número conocido de iteraciones  ->  for
Condición de parada               ->  while
```

No es una regla absoluta, pero sí un buen modelo mental inicial.

La trampa de range()

Python excluye el límite superior de range. Por tanto range(1, N) produce 1, 2, ..., N-1, no N.

```
print(list(range(1, N)))
```

Durante el desarrollo, imprimir un range como lista es una forma rápida de comprobar exactamente qué índices se recorrerán.

5. Algoritmo antes del código

```
Definir N
Validar N
Inicializar acumulador

PARA cada entero desde 1 hasta N:
    calcular el cuadrado
    sumar el cuadrado al acumulador

Mostrar el acumulador
```

No añadas ramas ni abstracciones que el problema no necesita. El objetivo del EX05 es comprender el bucle en sí mismo.

6. Traza el bucle mientras se ejecuta

Imprime temporalmente el contador y el acumulador dentro del bucle. Es una forma sencilla de depuración por trazado.

```
i = 1 | total = 1
i = 2 | total = 5
i = 3 | total = 14
i = 4 | total = 30
```

Utiliza también el debugger de VS Code: coloca un breakpoint dentro del bucle y pulsa F10 repetidamente. Observa cómo cambian i y total en el panel Variables.

La asignación no es una igualdad matemática

Una instrucción como total = total + i**2 no significa que total sea igual a sí mismo más algo. Significa: leer el valor anterior, calcular uno nuevo y volver a almacenarlo.

$$total_{new} = total_{old} + i^2$$

Python también ofrece la forma compacta total += i**2, que expresa la misma actualización.

7. Valida el resultado iterativo

Después del bucle, compara el resultado con la expresión analítica. Utilizamos la fórmula únicamente como solución de referencia:

$$S_{exact} = N(N+1)(2N+1) / 6$$

El hábito de ingeniería importante es: resultado numérico/algorítmico frente a una referencia conocida.

Casos de prueba requeridos

N	S esperada	Objetivo
1	1	caso normal mínimo
4	30	trazado manual
10	385	comparar con fórmula
0	política a decidir	caso límite
-5	rechazar	entrada inválida

8. Seguimiento del aprendizaje - EX05

Concepto	Qué debe comprenderse después del EX05
Iteración	Repetir una misma operación sobre una secuencia de valores.
Contador	Saber qué iteración/valor se está procesando.
Acumulador	Almacenar un resultado parcial que evoluciona durante el bucle.
Estado	Comprender que el valor actual depende de iteraciones anteriores.
for vs while	Usar for de forma natural cuando se conoce el número de pasadas; while cuando manda una condición de parada.
range()	Recordar que el límite superior queda excluido.
Trazado de debug	Observar cómo evolucionan i y total en cada iteración.
Validación	Comparar el resultado algorítmico con una referencia analítica.

Siguiente paso: el EX06 cambia el acumulador de aditivo a multiplicativo implementando N!.